

Status

Proposed

Context

S2PNexus must exchange data with a wide range of external systems: ERPs such as SAP, Oracle, Microsoft Dynamics, and Workday; supplier networks; external validation providers such as sanctions screening and tax ID validation; banking and payment systems; and customer-owned systems through custom integrations.

ADR-0002 and ADR-0003 assume business objects such as Supplier, Purchase Requisition, Purchase Order, and Forms exist within S2PNexus, but neither defines how those objects synchronize with the outside world reliably, idempotently, and at enterprise scale. Without a defined integration platform, every ERP connector becomes bespoke point-to-point code with inconsistent retry, error handling, and audit behavior, which is a significant operational and reliability risk for enterprise customers.

Decision

Implement a Universal Integration Platform as a core service providing standardized inbound and outbound integration patterns (REST, GraphQL, Webhooks, Event Bus), a library of certified ERP adapters, and platform-level guarantees for reliability, including retry, dead letter queue handling, idempotency, saga-based distributed transactions, and correlation tracing.

Integration Interfaces

- REST API: primary synchronous interface for CRUD operations on business objects; versioned and documented via OpenAPI
- GraphQL, optional: read-optimized interface for complex nested queries, for example supplier plus qualifications plus risk assessments in a single call; not a replacement for REST write operations
- Webhooks, outbound: tenant-configurable subscriptions to business events such as supplier.activated or po.approved, with signed payloads and retry with backoff
- Event Bus, internal and external: domain events published for all state transitions on core business objects; internal consumers such as the Approval Engine and Notification Engine, and external consumers such as customer data lakes and ERPs, subscribe via the same backbone

ERP Adapters

System	Typical Integration Method
SAP (ECC / S/4HANA)	IDoc, BAPI/RFC, or OData depending on version
Oracle (Fusion / EBS)	REST/SOAP integration layer
Microsoft Dynamics 365	Dataverse / OData connectors
Workday	RaaS, SOAP, or Workday Studio integration patterns

Adapter Architecture

- Each ERP adapter implements a common internal contract mapping external schema to and from the S2PNexus canonical model, so core platform logic never depends on ERP-specific formats
- Adapters must respect custom fields defined via the Metadata Engine, ADR-0004, without code changes
- New ERPs or providers are onboarded as adapters rather than platform rewrites

Inbound and Outbound Patterns

- Inbound: an external system pushes data into S2PNexus, for example an ERP vendor master sync, via REST, file-based batch such as SFTP or CSV, or event subscription
- Outbound: S2PNexus pushes data out, for example an approved PO to an ERP, via REST call, webhook, or event publication
- Both batch and real-time patterns are supported; batch jobs must be resumable and idempotent

Reliability and Delivery Guarantees

- Retry policy: exponential backoff with a configurable maximum attempt count, overridable per integration
- Dead Letter Queue: failed messages that exceed the retry limit land in a DLQ with full context for manual replay or alerting
- Idempotency: all inbound and outbound operations require an idempotency key so retries and duplicate deliveries do not create duplicate records, such as a duplicate supplier or purchase order
- Saga pattern: multi-step cross-system transactions, for example Supplier Activation followed by ERP Vendor Creation followed by Notification, use choreography or orchestration sagas with compensating actions on failure rather than distributed two-phase commit
- Correlation IDs: every inbound and outbound message, and every downstream event, carries a correlation ID and trace ID for end-to-end tracing across S2PNexus and external systems
- The event bus provides at-least-once delivery semantics; consumers must be idempotent

External Validation Providers

- Pluggable provider model for third-party validation and enrichment: sanctions and watchlist screening such as OFAC or EU lists, tax ID and VAT validation, address verification, and company enrichment
- Providers are integrated via the same adapter contract as ERPs, and are swappable per tenant without core platform changes
- Validation results are stored with source, timestamp, and confidence or expiry, since screening results expire and must be re-run periodically

Architecture Principles

- Canonical data model: all adapters map to and from a single internal canonical schema, which is metadata-extensible per ADR-0004; no adapter talks directly to another adapter
- No integration logic is embedded in business domain code such as Supplier or PO; the integration platform is a separate reusable layer, consistent with the reusable platform capability pattern established in ADR-0002 and ADR-0003
- All integrations are observable: every call is logged with request, response, latency, and status for audit and troubleshooting
- Security: all inbound APIs are authenticated via OAuth2 or API key, and all outbound webhooks are signed; see ADR-0006

Non-Functional Requirements

- API availability target and rate limits, defined per tenant
- Event bus delivery latency target for internal consumers
- DLQ retention period and replay tooling
- Backward-compatible API versioning policy: no breaking changes without a version bump and a deprecation window

Consequences

Positive: eliminates bespoke point-to-point ERP integrations, enables predictable, auditable, and replayable data exchange, and positions S2PNexus to onboard new ERPs and providers as adapters rather than rewrites.

Negative: the adapter framework and canonical model add upfront design cost. Eventual consistency delivered via sagas and the event bus requires business processes to tolerate brief data lag between S2PNexus and connected ERPs, which must be communicated to customers. The DLQ and retry infrastructure require ongoing operational monitoring investment.

Alternatives Considered

- Point-to-point custom integrations per customer: rejected, as this does not scale and carries a high maintenance burden with inconsistent reliability
- Synchronous distributed transactions, such as two-phase commit, across S2PNexus and ERPs: rejected, as ERPs rarely support distributed transaction coordination and this tightly couples the availability of both systems
- A single integration protocol only, REST without an event bus: rejected, as it does not efficiently support real-time downstream consumers such as the Approval Engine and Notification Engine at scale

Dependencies and Related ADRs

- ADR-0002: approval-driven state transitions, such as PO approval, are common outbound integration triggers
- ADR-0003: form and questionnaire responses may need to sync to external systems, such as compliance data flowing to a GRC tool
- ADR-0004: the canonical model must expose custom and metadata-extended fields through the same API and adapter contract
- ADR-0006: authentication, encryption, and audit requirements apply to all integration traffic

Open Questions

- Which ERP adapter ships first, and what is the certification and testing process for new adapters?
- Do we support customer-built custom adapters via an SDK, or only platform-certified adapters?
- What is the maximum acceptable data lag between an event occurring and it reaching an external consumer?